Semana 13, explicada fácil

La base de datos y la persistencia del sistema de reservas

MareaDorada Pro v1.0 · Corte vertical “Crear una reserva”



Integrantes

Erick Genaldo Flores

Juan David Arroyave

Arquitectura de Sistemas I · Docente: Ing. Elías Buitrago Bolivar

Universidad Central · Facultad de Ingeniería · 2026-2

Contenido

1. ¿De qué se trató la semana 13? **2**

2. Los dos requisitos que elegimos **2**

3. Qué tan bien debe funcionar **3**

4. La base de datos **4**

4.1. ¿Qué guardamos? 4

4.2. ¿Por qué MongoDB y no PostgreSQL? 4

4.3. El truco para evitar dobles reservas 5

4.4. Otras protecciones de la base 5

4.5. Los datos de prueba 5

5. Cómo está organizado el código **6**

5.1. Las capas 6

5.2. Los repositorios 6

5.3. La transacción: todo o nada 7

6. Las pruebas **7**

7. Cómo mostrarlo funcionando **8**

8. Preguntas que podría hacer el profesor **8**

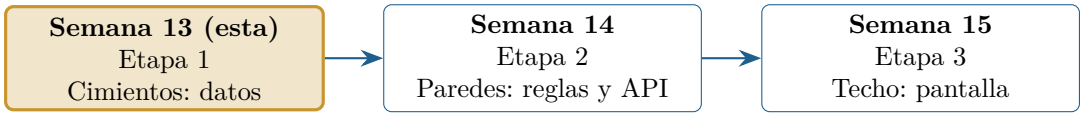
9. Qué sigue **9**

Mini glosario **10**

1. ¿De qué se trató la semana 13?

El proyecto es un sistema para que la recepción del resort **Marea Dorada** haga reservas de habitaciones. Este semestre no se construye todo el sistema, sino un **corte vertical**: una sola función completa, de punta a punta. La función elegida es **"Crear una reserva"**.

Ese corte se construye en tres semanas del curso, como una casa:



Ojo

Las guías del profesor hablan de "semana 1, 2 y 3" para referirse a las *etapas* del prototipo. En el calendario del curso son las semanas **13, 14 y 15**. Cuando el README dice "semana 2", significa la semana 14 del curso.

En la semana 13 hicimos los **cimientos**: decidir qué se construye primero, definir qué tan bien debe funcionar, preparar la base de datos y comprobar, con pruebas reales, que guardar y leer reservas funciona bien. Todavía no hay pantalla ni botones; eso viene después.

En una frase

Dejamos lista y probada la parte del sistema que *guarda y lee* las reservas, para que en las siguientes semanas solo haya que construir encima.

Concretamente, la guía pedía cuatro cosas, y cada una tiene su sección en este documento:

1. Elegir **dos requisitos funcionales** y decidir cuál va primero (sección 2).
2. Definir **requisitos de calidad medibles** para esos dos (sección 3).
3. Preparar una **base de datos con datos de prueba** (sección 4).
4. Programar la **persistencia** y **probarla de verdad** (secciones 5 y 6).

2. Los dos requisitos que elegimos

Un **requisito funcional (RF)** es algo que el sistema *hace*. Del documento del segundo corte escogimos estos dos:

Orden	Requisito	Qué hace, en simple
1	RF-04 Crear reserva	El recepcionista dice quién es el huésped, qué tipo de habitación quiere, las fechas y cuántas personas van. El sistema le asigna una habitación libre, calcula el total y le da un número de reserva. Si no se puede, le dice por qué.
2	RF-02 Consultar disponibilidad	El recepcionista pregunta "¿qué Suites hay libres del 21 al 22 de diciembre?" y el sistema le muestra la lista.

¿Por qué crear reserva va primero?

Los dos son muy útiles para recepción, así que el desempate fue por otros criterios:

- **Es lo más riesgoso.** El problema real del resort son las *dobles reservas*: dos personas que terminan con la misma habitación. Eso pasa al crear reservas, no al consultar. Lo más riesgoso conviene atacarlo primero, cuando todavía hay tiempo de corregir.
- **Uno incluye al otro.** Para crear una reserva, el sistema primero tiene que buscar habitaciones libres, que es justo lo que hace consultar disponibilidad. Al construir el primero, el segundo queda casi hecho.

Para entenderlo: el orden

Es como aprender a manejar: primero se practica lo peligroso (frenar a tiempo) y no lo cómodo (poner música). Y de paso, para frenar bien ya aprendiste a mirar la vía, que es lo que necesitas para todo lo demás.

Lo que quedó por fuera (a propósito): iniciar sesión, registrar huéspedes nuevos (usamos huéspedes ya cargados), modificar o cancelar reservas, pagos, reportes y app móvil. Así el corte es pequeño y se puede terminar.

3. Qué tan bien debe funcionar

Un **requisito no funcional (RNF)** no dice *qué* hace el sistema, sino *qué tan bien* lo hace: si es rápido, si es confiable, si es fácil de mantener. La guía pide que cada uno se pueda **medir**: no vale decir "debe ser rápido", hay que decir "debe responder en menos de tanto".

Para clasificarlos usamos la norma **ISO/IEC 25010:2023**, que es como un "catálogo oficial" de tipos de calidad del software. Elegimos cuatro.

Algo importante: en la semana 13 solo existe la parte de la base de datos, así que cada RNF se comprobó **hasta donde se puede hoy**. El resto se mide cuando existan la API (semana 14) y la pantalla (semana 15). Las guías piden decir con claridad qué está verificado y qué está pendiente.

RNF	Qué pide, en simple	Qué comprobamos en la semana 13	Estado
RNF-01 Rapidez	Buscar habitaciones libres debe ser rápido, aunque haya muchas reservas.	Con más de 5 000 reservas, 20 búsquedas directas a la base. Límite: 200 ms cada una. La más lenta tardó unos 3 ms.	**Cumple** en la base. **Pendiente** (sem. 14): medirlo por la API con 10 usuarios a la vez; meta: 2 segundos.
RNF-03 Todo o nada	Si algo falla a mitad de una reserva, no puede quedar nada guardado a medias.	Provocamos un error justo después de guardar. No quedó nada guardado.	**Cumple** en la base. **Pendiente** (sem. 14): repetirlo con el servicio.
RNF-06 Orden del código	Solo una parte del código habla con la base, y los servicios se pueden probar sin ella.	Un programa revisó todos los archivos: 0 accesos a la base fuera de su lugar.	**Cumple** la separación. **Pendiente** (sem. 14): probar los servicios con una base de mentira.
RNF-08 Muchos a la vez	Si 10 personas reservan la misma habitación al mismo tiempo, solo una lo logra.	Lanzamos 10 intentos al mismo tiempo: 1 reserva guardada y 9 rechazadas.	**Cumple** en la base. **Pendiente** (sem. 15): repetirlo de punta a punta.

Ojo

Regla de negocio no es lo mismo que RNF. "No se puede reservar una habitación ocupada" es una *regla del negocio*: describe cómo funciona el hotel. En cambio, "con 10 personas al tiempo, solo una gana" es un *RNF*: describe qué tan confiable es el sistema. El profesor insiste en no mezclarlos.

Cómo decirlo en clase

"Elegimos cuatro requisitos de calidad de la ISO 25010: rapidez, que no queden datos a medias, que el código esté ordenado por capas y que funcione bien cuando varias personas reservan a la vez. Cada uno tiene un número límite y una prueba. En esta semana los cuatro se cumplen en la parte de la base de datos; lo que depende de la API y de la pantalla queda marcado como pendiente para las semanas 14 y 15."

4. La base de datos

4.1. ¿Qué guardamos?

La base de datos es donde queda guardada la información aunque se apague el computador. Usamos **MongoDB**, que guarda la información en **colecciones** (piensa en cajones de un archivador), y cada cajón tiene **documentos** (las fichas).

Cajón (colección)	Qué guarda
`tipos_habitacion`	Los tres tipos: Estándar, Familiar y Suite, con cuántas personas caben y cuánto cuesta la noche.
`habitaciones`	Las 24 habitaciones del resort, cada una con su número y su tipo.
`huespedes`	Los datos de los huéspedes: nombre, documento, teléfono, correo.
`reservas`	Cada reserva: quién, qué habitación, qué fechas, cuántas personas, cuánto cuesta y su estado.
`contadores`	Un contador para darle a cada reserva un número consecutivo (1, 2, 3...).

4.2. ¿Por qué MongoDB y no PostgreSQL?

En el documento del segundo corte habíamos dicho que íbamos a usar Java con PostgreSQL. Lo cambiamos porque **el proyecto ya estaba hecho en Python con MongoDB**. Tener dos bases de datos distintas en el mismo sistema solo complica las cosas. Además, MongoDB permite *transacciones* (lo de "todo o nada" que se explica en la sección 5), que era lo que necesitábamos.

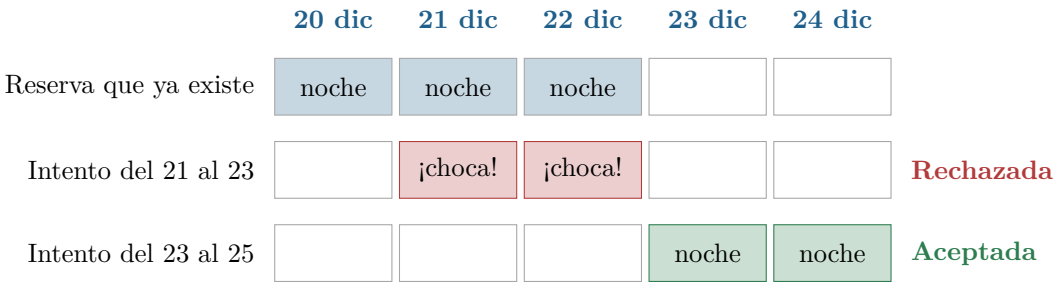
Ojo

Para que MongoDB permita transacciones tiene que estar en un modo especial llamado **replica set**. Por eso, para correr el proyecto se usa Docker, que ya lo deja configurado así. Si alguien intenta correrlo sin ese modo, el programa avisa con un mensaje claro.

4.3. El truco para evitar dobles reservas

Esta es la parte más importante y vale la pena entenderla bien. Cada reserva guarda la **lista de noches** que ocupa. Por ejemplo, una reserva del 20 al 23 de diciembre ocupa las noches del 20, 21 y 22. El 23 no cuenta como noche, porque ese día el huésped se va.

Luego le pusimos a la base una regla que dice: *"la misma habitación no puede tener dos reservas activas con la misma noche"*. A eso se le llama **índice único**. Mira cómo funciona con la Suite 301:



El segundo intento comparte las noches del 21 y el 22, así que la base lo rechaza. El tercero empieza el 23, justo el día en que el otro huésped se va, así que no comparte ninguna noche y se acepta.

Para entenderlo: el índice único

Es como el registro de puestos en un parqueadero con cuadrícula por día: si el puesto 301 ya tiene marcada la casilla del 21, nadie más puede marcar esa misma casilla. No importa si llegan dos personas al mismo tiempo: la casilla solo se puede marcar una vez.

Lo bueno de que esta regla la cumpla **la propia base de datos** es que funciona aunque lleguen muchas solicitudes al mismo tiempo. Por eso la prueba de las 10 reservas simultáneas da exactamente una ganadora.

4.4. Otras protecciones de la base

- **Validadores:** la base rechaza datos sin sentido, por ejemplo una fecha de salida anterior a la de entrada, o un precio negativo.
- **Comprobar que existan:** antes de guardar, el programa verifica que el huésped y la habitación existan de verdad. (En PostgreSQL esto lo hacían las "claves foráneas"; MongoDB no las tiene, así que lo hace el código.)
- **Las reservas canceladas no bloquean:** la regla de las noches solo cuenta las reservas confirmadas o en curso.

4.5. Los datos de prueba

Para probar necesitamos datos que siempre empiecen igual, como un tablero de ajedrez que se arma en la misma posición antes de cada partida:

- 24 habitaciones: Estándar 101 a 112, Familiar 201 a 208 y Suite 301 a 304.
- 3 huéspedes: Ana Torres, Carlos Méndez y Laura Smith.
- 3 reservas preparadas a propósito:

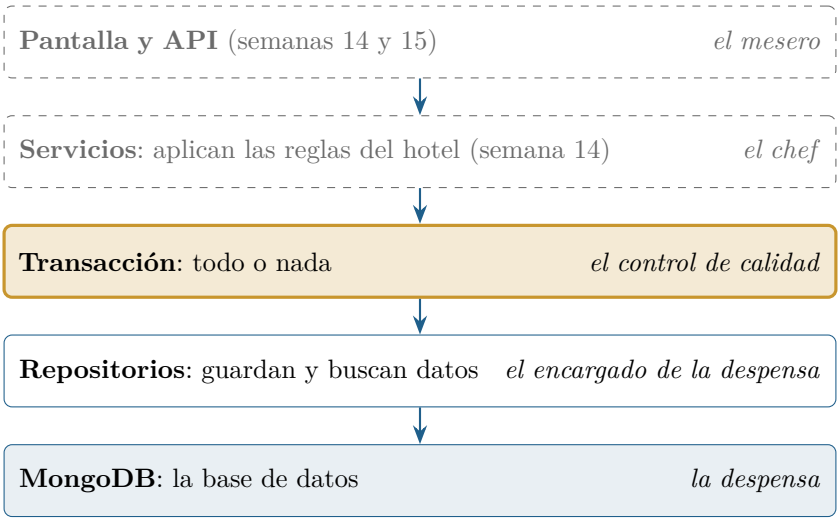
- Suite **301**, del 20 al 23 de diciembre, **confirmada**: sirve para probar choques.
- Suite **302**, mismas fechas, **cancelada**: sirve para probar que una cancelada no estorba.
- Estándar **101**, del 10 al 12 de diciembre, **confirmada**.

Un solo comando (`reset_db`) borra todo y lo deja otra vez en ese estado. Además usa una base aparte (`marea_test`), así que nunca toca la base real del hotel.

5. Cómo está organizado el código

5.1. Las capas

El código está dividido en **capas**, y cada una tiene un solo trabajo. La regla de oro es que **solo la capa de repositorios habla con la base de datos**.



Lo punteado es lo que todavía no existe. En la semana 13 construimos de la transacción hacia abajo.

Para entenderlo: las capas

Piensa en un restaurante. El mesero (la pantalla) no entra a la despensa: le pasa el pedido al chef. El chef (los servicios) decide cómo se prepara, pero no sabe dónde está guardado cada ingrediente: se lo pide al encargado de la despensa (los repositorios). Si mañana cambian la despensa de lugar, solo el encargado tiene que aprender dónde quedó; el mesero y el chef siguen igual.

¿Para qué sirve esto? Si mañana cambiáramos MongoDB por otra base de datos, solo habría que cambiar los repositorios. Y en la semana 14 se pueden probar los servicios con una "despensa de mentira", sin necesitar la base real.

5.2. Los repositorios

Hay uno por cajón. Cada uno solo sabe buscar y guardar; **no decide reglas**:

- **Tipos de habitación:** buscar un tipo para saber su capacidad y su precio.
- **Habitaciones:** buscar las habitaciones libres de un tipo en unas fechas.

- **Huéspedes:** buscar un huésped para saber si existe.
- **Reservas:** guardar una reserva, buscarla por su número y ver si una habitación ya tiene reservas en ciertas fechas.

Si algo no existe, el repositorio devuelve "nada" (en Python, `None`) en vez de romperse. Así el servicio puede responder con calma "ese huésped no existe".

5.3. La transacción: todo o nada

Una **transacción** agrupa varios pasos para que se hagan *todos* o *ninguno*. Al crear una reserva pasan varias cosas: se verifica el huésped, se verifica la habitación, se aumenta el contador y se guarda la reserva. Si cualquiera falla, se deshace todo como si nunca hubiera pasado.

Para entenderlo: la transacción

Es como una transferencia bancaria: sale plata de una cuenta y entra en otra. Si el sistema se cae en la mitad, el banco no puede quedarse con la plata "en el aire". O se hacen los dos movimientos, o ninguno.

Además, si dos personas chocan al mismo tiempo, el sistema **reintenta solo**. Así, en la prueba de las 10 personas, una gana y las otras 9 reciben un rechazo claro, sin errores raros.

6. Las pruebas

Escribimos **15 pruebas automáticas** que se conectan a una base MongoDB de verdad (no a una simulación). Antes de cada prueba la base vuelve a su estado inicial, para que ninguna afecte a la otra. Las 15 pasan.

Qué probamos	**Qué pasó**
Guardar una reserva y leerla desde una conexión nueva	Quedó guardada de verdad: número 4, confirmada, total $600 000.
Buscar una reserva que no existe	El sistema responde "no existe" sin romperse.
Suites libres del 21 al 22 de diciembre	Salen la 302, la 303 y la 304. La 301 no, porque está ocupada.
Reservar la 301 desde el 23	Se permite: el día de salida no cuenta como noche.
Reservar la 301 en fechas que se cruzan	La base lo rechaza.
Fecha de salida antes de la de entrada	La base lo rechaza.
Huésped o habitación que no existe	El sistema lo rechaza.
Un error a mitad de la reserva	No queda nada guardado y el contador no avanza.
Rechazo y nuevo intento con otra habitación	El segundo intento funciona sin problema.
10 personas reservando lo mismo al mismo tiempo	1 reserva guardada y 9 rechazadas.
Velocidad con más de 5 000 reservas	La búsqueda más lenta tardó unos 3 ms (el límite era 200 ms).

Todos los resultados quedaron guardados en la carpeta `evidencias/` del proyecto, junto con capturas de pantalla.

Cómo decirlo en clase

"No nos quedamos en que el código compila: lo probamos contra una base real. Guardamos una reserva y la leímos desde otra conexión para confirmar que de verdad quedó guardada. Y lo más importante: lanzamos 10 reservas de la misma habitación al mismo tiempo y solo una quedó. Eso demuestra que el sistema evita las dobles reservas, que era el problema principal del resort."

7. Cómo mostrarlo funcionando

Si el profesor quiere verlo, estos son los comandos (desde la carpeta del proyecto, en la rama `semana13-persistencia`):

```
docker compose up -d
source venv/bin/activate
python -m scripts.reset_db
python -m pytest -v tests/
python -m scripts.demo_persistencia
```

Qué se ve con cada uno:

1. `docker compose up -d` enciende MongoDB.
2. `reset_db` arma la base y muestra: 12 Estándar, 8 Familiares, 4 Suites, 3 huéspedes.
3. `pytest` corre las pruebas y al final dice **15 passed**.
4. `demo_persistencia` muestra en cinco líneas: habitaciones libres, una reserva guardada, la misma reserva leída de nuevo, una búsqueda de algo que no existe y un choque rechazado.

8. Preguntas que podría hacer el profesor

¿Por qué no usaron PostgreSQL, como decía el documento del segundo corte?
Porque el proyecto ya estaba hecho en Python con MongoDB. Usar dos bases en el mismo sistema era más complicado. MongoDB también permite transacciones, y la regla contra dobles reservas la logramos con un índice único sobre las noches. El cambio quedó justificado en el README.

¿Qué pasa si dos recepcionistas reservan la misma habitación al mismo tiempo?
Solo una reserva queda guardada, porque la base no permite dos reservas activas con la misma noche en la misma habitación. Lo probamos con 10 intentos simultáneos.

¿Dónde están las reglas del negocio?
En la semana 13 todavía no: los repositorios solo guardan y buscan. Las reglas (capacidad, fechas válidas, que exista el huésped) las aplicarán los servicios en la semana 14. La base solo *refuerza* las más críticas como seguro.

¿Entonces ya cumplen todos los RNF?
En la parte de la base de datos, sí, y lo medimos. Pero varios RNF hablan del sistema completo: por ejemplo, la rapidez se debe medir también por la API con 10 usuarios a la vez. Eso queda pendiente para las semanas 14 y 15, y así está marcado en la matriz. No lo damos por cumplido antes de medirlo.

¿Cómo saben que los datos quedaron guardados de verdad?
Porque después de guardar abrimos una conexión completamente nueva y desde ahí leímos la reserva. Si solo hubiera quedado en memoria, no aparecería.

¿Por qué la reserva cancelada no bloquea la habitación?
Porque la regla de las noches solo cuenta las reservas confirmadas o en curso. Lo comprobamos con la Suite 302, que tiene una reserva cancelada y aun así aparece libre.

¿Qué es un replica set y por qué lo necesitan?
Es un modo de funcionamiento de MongoDB. Solo en ese modo se pueden hacer transacciones (el "todo o nada"). Para nosotros basta con un solo servidor en ese modo, y Docker lo configura solo.

9. Qué sigue

Lo de la semana 13 es la base sobre la que se construyen las dos semanas siguientes. Esto es lo que piden las guías del profesor, aplicado a nuestro proyecto.

Semana 14 (etapa 2): las reglas y la API

- **Los servicios** (el chef): uno para consultar disponibilidad y otro para crear reservas. El de crear reservas revisa que el huésped exista, que las fechas tengan sentido y no estén en el pasado, y que no se pase la capacidad. Después busca la habitación libre de menor número, calcula el total y guarda la reserva.
- **La API** (las "puertas" por las que la pantalla le habla al sistema):
  - Consultar disponibilidad: responde la lista de habitaciones libres, aunque venga vacía.
  - Crear reserva: responde **201** si se creó, **400** si los datos están mal, **404** si el huésped no existe y **409** si no hay habitaciones libres.
- **El contrato:** un documento que dice exactamente qué se envía y qué se recibe en cada puerta, para que la pantalla no tenga que adivinar.
- **Las pruebas:** un caso que funciona y un rechazo por cada RF, usando la API. Además, probar los servicios con una base de mentira (RNF-06) y medir la rapidez por la API con 10 usuarios a la vez (RNF-01).

Semana 15 (etapa 3): la pantalla

- **La pantalla "Nueva reserva":** primero se consulta qué habitaciones hay libres y luego se reserva; al final se muestra el número de reserva, la habitación y el total.
- **Que se porte bien:** mostrar "enviando…", no dejar enviar dos veces, conservar lo escrito si hay un error, dar mensajes claros y no decir "reserva exitosa" si se cayó la conexión.
- **El cierre:** probar todo desde la pantalla, marcar cada RNF como cumple, no cumple o no verificado, y preparar una demostración corta para el profesor.

Para entenderlo: cómo se conectan las tres semanas

La semana 13 llenó y organizó la despensa. En la semana 14 se contrata al chef, que sabe las recetas (las reglas). En la semana 15 llega el mesero, que toma el pedido del cliente (la pantalla). Ninguno puede trabajar sin el anterior.

Mini glosario

Corte vertical: una sola función del sistema, construida completa de punta a punta.

RF (requisito funcional): algo que el sistema hace. Ejemplo: crear una reserva.

RNF (requisito no funcional): qué tan bien lo hace. Ejemplo: que responda rápido.

ISO/IEC 25010: norma internacional que clasifica los tipos de calidad del software.

Persistencia: guardar la información para que no se pierda al apagar el sistema.

Colección: en MongoDB, un "cajón" donde se guardan documentos del mismo tipo.

Índice único: regla de la base que impide repetir un valor; aquí, la misma noche en la misma habitación.

Transacción: grupo de pasos que se hacen todos o ninguno.

Replica set: modo de MongoDB necesario para usar transacciones.

Repositorio: la parte del código encargada de guardar y buscar datos.

Prueba de integración: prueba que usa la base de datos real, no una simulación.

Docker: programa que enciende MongoDB ya configurado, con un solo comando.